UptimeRobot

CONF42 PLATFORM ENGINEERING 2026 · LIGHTNING TALK

Monitoring is still the one thing nobody automated

Making new services arrive already being watched, from the command line

Eleftheria Batsou
Developer Advocate, UptimeRobot · @BatsouElef



UptimeRobot

Your golden path, today



Scaffold

one command

✓ automated



CI / CD

templated

✓ automated



Secrets

injected

✓ automated



Infrastructure

as code

✓ automated



Dashboards

auto-wired

✓ automated



Monitoring

a human, later, by hand

✗ if anyone remembers

Every step is a default, except the one that tells you the service is down.

UptimeRobot

What the gap looks like in production

No monitor

Runs for months. The first alert is a customer support ticket.

no data

Wrong monitor

Checks the homepage, not /healthz. Pages someone who left the team.



Orphaned monitor

Service renamed or retired. The check stays, pointing at nothing.



Three symptoms. One root cause.

UptimeRobot

Not a tooling problem.
A workflow problem.

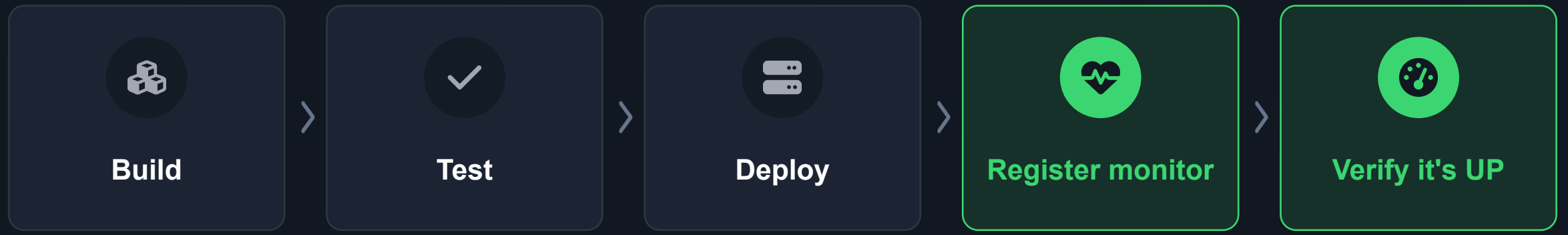
Monitoring sits outside the golden path, so it depends on **discipline instead of defaults.**

50 new services a year means 50 separate acts of remembering

1 default on the golden path covers all of them

UptimeRobot

Move monitoring inside the deploy



The deploy isn't done until the service is being watched.

The pipeline already knows everything a human would type into the web form:

`$SERVICE_NAME` `$HEALTH_URL` `$OWNER_TEAM` `$ENV` `$TIER`

UptimeRobot

The UptimeRobot CLI in 30 seconds

1 Install

```
$ npm install -g @uptimerobot/cli
$ uptimerobot --version
```

Official, open source, API v3. Node 22.12+

2 Authenticate

```
# laptop: key goes to OS keychain
$ uptimerobot auth login

# CI: from your secret store
$ export UPTIMEROBOT_API_KEY=…
```

Flag › env var › stored key

3 Create

```
$ uptimerobot monitors \
    create http \
  --name checkout-api \
  --url https://…/healthz \
  --interval 60 \
  --timeout 30
```

Remember that last flag

Also covers: `monitors` `alert-contacts` `integrations` `maintenance-windows` `monitor-groups` `status-pages` `incidents` `tags`

UptimeRobot

Demo: validate before you touch prod

terminal

```
$ uptimerobot monitors create http \
  --name "checkout-api (prod)" \
  --url https://checkout.example.com/healthz \
  --interval 60 --timeout 30 \
  --method GET \
  --tag-names team:payments \
  --tag-names tier:1 \
  --assigned-alert-contacts 12345 \
  --dry-run
```

No credentials, no network. Run it as a PR check.

stdout · exit 0


```
{
  "dryRun": true,
  "method": "POST",
  "path": "/v3/monitors",
  "body": {
    "type": "HTTP",
    "friendlyName": "checkout-api (prod)",
    "url": "https://checkout.example.com/healthz",
    "interval": 60, "timeout": 30,
    "httpMethodType": "GET",
    "tagNames": ["team:payments", "tier:1"],
    "assignedAlertContacts": [{ "alertContactId": 12345 … }]
  }
}
```

UptimeRobot

Template vs. per-service

In the platform template

decided once, by the platform team

Interval per tier	`--interval`
Timeout	`--timeout`
HTTP method	`--method GET`
Success codes	`--success-http-response-codes`
SSL / domain expiry	`--ssl-expiration-reminder`
Tag convention	`team:* tier:* env:*`
Team → alert contacts	`--assigned-alert-contacts`

Supplied by each service

from the service's own metadata

Name + environment	`"checkout-api (prod)"`
Health endpoint	`/healthz`
Owning team	`team:payments`
Tier	`tier:1`
Job schedule	`heartbeat interval`

Copying the same value into every service? It belongs on the left.

Make the step safe to re-run

deploy/register-monitor.sh

```
NAME="$SERVICE_NAME ($ENV)"

# look it up first (name filter is a partial match)
ID=$(uptimerobot monitors list --name "$NAME" \
      --format plain --columns id,friendlyName \
    | awk -F'\t' -v n="$NAME" '$2==n {print $1}')

if [ -z "$ID" ]; then
  uptimerobot monitors create http --name "$NAME" \
    --url "$HEALTH_URL" --interval "$INTERVAL" \
    --timeout 30 --method GET --tag-names "team:$TEAM"
else
  uptimerobot monitors update "$ID" \
    --url "$HEALTH_URL" --interval "$INTERVAL"
fi
```

No upsert

"create" on every deploy means a new duplicate monitor on every deploy.

No retry on writes

The CLI retries GET, never POST / PATCH / DELETE. Keep the IDs it returns.

Predictable names

service + environment. Match the exact name yourself.

Details that bite the first time

Credentials

Writes need the Main API key. A read-only key gets 403 (exit 5). Keep two keys; give CI the right one.

```
read-only ≠ provisioning
```

Required flags

HTTP monitors need --interval and --timeout. Everyone forgets --timeout.

```
timeout is required. Pass --timeout or --set.
```

Plan limits

Minimum interval depends on the plan: 5 min on Free. Don't hard-code 30 s in a shared template.

```
interval ≥ plan minimum
```

Silent defaults

No --method? HTTP monitors send HEAD. Many frameworks route HEAD differently.

```
--method GET
```

UptimeRobot

Exit codes your pipeline should read

0	**Success**	monitor registered	**continue**
2	**Invalid input**	config bug: missing flag, bad value	**fail the build**
4	**Unauthenticated**	missing or wrong API key	**fail the build**
5	**Forbidden**	e.g. read-only key trying to write	**fail the build**
7	**Rate limited**	safe to retry with back-off	**retry, then warn**
8	**Conflict**	state changed underneath you	**re-read, then decide**

```
# errors → stderr, as JSON
{"error":{
 "code":"INVALID_INPUT",
 "path":"timeout", …}}
```

Block on config errors. Don't block a release on a rate limit.

UptimeRobot

Not everything has a URL: heartbeats

Cron job / worker runs

Calls its heartbeat URL

Missed it? Alert

terminal

```
$ uptimerobot monitors create heartbeat \
    --name "nightly-billing-export (prod)" \
    --interval 86400 \
    --grace-period 1800
```

The ping URL is a secret

Redacted in CLI output by default. Reveal it only in the step that writes it to the job's secret store:

```
--reveal-secrets
```



When to reach for something other than a CLI

CLI in the pipeline

Per-service monitors created with the deploy

Terraform provider

The whole fleet as desired state, with drift detection and review

MCP server

AI agents investigating incidents, no shelling out

Dashboard

A human at 3 a.m., figuring out what broke

Pick one source of truth per monitor. Pipeline-created monitors edited by hand in the UI = drift.

UptimeRobot

What you can't template

It depends on things your service doesn't own.



Third-party dependencies

UptimeRobot already watches 6,300+ services. A template just can’t know which ones your service relies on.



Cross-service user journeys

Checkout works only if four services work



Alert routing

Who gets woken up depends on people and rotations, not code



Thresholds

Sensible response-time limits need real traffic. Day-one services have none

Template the baseline on day one. Make these a conscious decision, not a forgotten one.

UptimeRobot

Your golden path, fixed

- ✓ Repo scaffold
- ✓ CI / CD
- ✓ Secrets
- ✓ Infrastructure
- ✓ Dashboards
- ✓ Monitoring

1. **Monitoring is a deploy step**
   not a follow-up ticket
2. **Template the policy**
   let each service supply its identity
3. **Safe to re-run**
   right key, explicit flags, look up before create
4. **Humans for what you don't own**
   third parties, journeys, routing, thresholds

UptimeRobot

Thank you!

New services should arrive **already being watched.**

`github.com/uptimerobot/uptimerobot-cli`

`npm install -g @uptimerobot/cli`

uptimerobot.com

@BatsouElef

